

Exercises and Solutions

A Brief Introduction to Programming with R

Ana Armendariz & Federica Mascolo

2026-09-04

Table of contents

Part I: Background and tools	2
Exercise a: setting up a working environment	2
Exercise b: R scripts	2
Part II: First steps and basic concepts	2
Exercise a: write a sum function	2
Exercise b: robustness and warnings	3
Exercise c: standard deviation function	3
Exercise d: standard error function	4
Exercise e: t-test	4
Exercise f: Fibonacci sequence	5
Exercise g: multiples of 3 and 5 (*)	5
Exercise h: prime numbers (*)	6
Part III: Working with data	6
Exercise i: working with data using dplyr	6
Exercise i.1	6
Exercise i.2	7
Exercise i.3	8
Exercise i.4	9
Exercise i.5	9
Exercise i.6	10
Exercise i.7	10
Exercise j: visualizing data	11
Exercise j.1	11
Exercise j.2	11
Exercise j.3	12
Exercise j.4	12
Exercise j.5	13
Exercise j.6	14
Exercise j.7	14

```
install.packages(c("dplyr", "ggplot2"))
library(dplyr)
library(ggplot2)
```

Part I: Background and tools

Exercise a: setting up a working environment

1. Open RStudio and navigate to your desired working folder
2. Create a new folder called `r_course`
3. Set it as your working directory
4. Create subfolders: `data` and `code`
5. (*) Create a new R Project in the `r_course` folder

Solution

```
dir.create("r_course")
setwd("r_course")
dir.create("data")
dir.create("code")
getwd()
```

R Project: File > New Project > Existing Directory, then pick `r_course`.

Exercise b: R scripts

1. In the R console, type `print("Hello world")`
2. Create a new R script: File > New File > R Script (or Ctrl+Shift+N)
3. Type the same code in the script and run it using Ctrl+Enter

Solution

```
print("Hello world")
```

```
[1] "Hello world"
```

Part II: First steps and basic concepts

Exercise a: write a sum function

Write a function that takes a numeric vector as input and returns the sum of the vector's elements. Test by comparing with built-in `sum()` function.

Solution

```
my_sum <- function(x) {
  total_sum <- 0

  for (value in x) {
    total_sum <- total_sum + value
  }

  return(total_sum)
}

numeric_vector <- c(1, 5, 4, 3)
my_sum(numeric_vector)
```

```
[1] 13
```

```
sum(numeric_vector)
```

```
[1] 13
```

Exercise b: robustness and warnings

Test your `my_sum()` function with `numbers2 <- c("1", "2", "3")`. Add error checking to make the function robust.

Solution

```
my_sum <- function(x) {  
  if (!is.numeric(x)) {  
    warning("The input is not numeric; attempting to convert it.")  
    x <- suppressWarnings(as.numeric(x))  
  }  
  
  if (anyNA(x)) {  
    stop("The input contains values that cannot be converted to numbers.")  
  }  
  
  total_sum <- 0  
  
  for (value in x) {  
    total_sum <- total_sum + value  
  }  
  
  return(total_sum)  
}  
  
numbers2 <- c("1", "2", "3")  
my_sum(numbers2)
```

```
[1] 6
```

Exercise c: standard deviation function

Implement a function to compute the standard deviation:

$$SD = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Solution

```
my_sd <- function(x) {  
  if (!is.numeric(x)) {  
    stop("x must be numeric.")  
  }  
  
  n <- length(x)  
  if (n < 2) {
```

```

    stop("x must contain at least two values.")
  }

  x_bar <- mean(x)
  squared_deviations <- (x - x_bar)^2
  sqrt(sum(squared_deviations) / (n - 1))
}

my_sd(numeric_vector)

```

```
[1] 1.707825
```

```
sd(numeric_vector)
```

```
[1] 1.707825
```

Exercise d: standard error function

Building on Exercise c, implement:

$$SE_{\bar{x}} = \frac{SD}{\sqrt{N}}$$

Solution

```

my_se <- function(x) {
  my_sd(x) / sqrt(length(x))
}

my_se(numeric_vector)

```

```
[1] 0.8539126
```

```
sd(numeric_vector) / sqrt(length(numeric_vector))
```

```
[1] 0.8539126
```

Exercise e: t-test

Implement a one-sample t-test function:

$$t = \frac{\bar{x} - \mu_0}{SE_{\bar{x}}}$$

Solution

```

my_t_test <- function(x, mu_0) {
  (mean(x) - mu_0) / my_se(x)
}

my_t_test(numeric_vector, mu_0 = 3)

```

```
[1] 0.29277
```

```
unname(t.test(numeric_vector, mu = 3)$statistic)
```

```
[1] 0.29277
```

Exercise f: Fibonacci sequence

Generate the first 30 Fibonacci numbers where $F_0 = 0$, $F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$ for $n > 1$.

Solution

```
fibonacci_sequence <- numeric(30)
fibonacci_sequence[1] <- 0
fibonacci_sequence[2] <- 1

for (i in 3:30) {
  fibonacci_sequence[i] <-
    fibonacci_sequence[i - 1] + fibonacci_sequence[i - 2]
}

fibonacci_sequence
```

```
[1]      0      1      1      2      3      5      8     13     21     34     55     89    144
[14]    233    377    610    987   1597   2584   4181   6765  10946  17711  28657  46368  75025
[27] 121393 196418 317811 514229
```

Exercise g: multiples of 3 and 5 (*)

Write a function that computes the sum of all multiples of 3 or 5 up to a number N . Multiples of both 3 and 5 should only be added once.

Solution

```
sum_multiples_3_or_5 <- function(N) {
  if (length(N) != 1 || !is.numeric(N) || is.na(N) || N < 1) {
    stop("N must be one positive number.")
  }

  total <- 0

  for (i in seq_len(floor(N))) {
    if (i %% 3 == 0 || i %% 5 == 0) {
      total <- total + i
    }
  }

  return(total)
}

sum_multiples_3_or_5(100)
```

```
[1] 2418
```

Exercise h: prime numbers (*)

Write a function that computes the sum of all prime numbers up to a number N .

Solution

```
is_prime <- function(x) {
  if (x < 2) {
    return(FALSE)
  }

  largest_possible_divisor <- floor(sqrt(x))

  if (largest_possible_divisor < 2) {
    return(TRUE)
  }

  !any(x %% seq.int(2, largest_possible_divisor) == 0)
}

sum_primes_up_to <- function(N) {
  if (length(N) != 1 || !is.numeric(N) || is.na(N) || N < 1) {
    stop("N must be one positive number.")
  }

  candidates <- seq_len(floor(N))
  prime_numbers <- candidates[
    vapply(candidates, is_prime, logical(1))
  ]

  sum(prime_numbers)
}

sum_primes_up_to(100)
```

```
[1] 1060
```

Part III: Working with data

Exercise i: working with data using dplyr

Exercise i.1

- Load the dataset with `data(swiss)`
- Use `mutate()` to create two new variables:
 - `UrbanizationRate`: the inverse of Agriculture ($100 - \text{Agriculture}$)
 - `FertilityCategory`: a factor variable with levels 0-60 “Low”, 60-80 “Medium”, 80-100 “High”
- Hint: use `case_when()` for categorization

Solution

```

data(swiss)

swiss_data <- swiss |>
  mutate(
    UrbanizationRate = 100 - Agriculture,
    FertilityCategory = case_when(
      Fertility <= 60 ~ "Low",
      Fertility > 60 & Fertility <= 80 ~ "Medium",
      Fertility > 80 & Fertility <= 100 ~ "High",
      TRUE ~ NA_character_
    ),
    FertilityCategory = factor(
      FertilityCategory,
      levels = c("Low", "Medium", "High")
    )
  )

head(swiss_data)

```

	Fertility	Agriculture	Examination	Education	Catholic	Infant.Mortality
Courtelary	80.2	17.0	15	12	9.96	22.2
Delemont	83.1	45.1	6	9	84.84	22.2
Franches-Mnt	92.5	39.7	5	5	93.40	20.2
Moutier	85.8	36.5	12	7	33.77	20.3
Neuveville	76.9	43.5	17	15	5.16	20.6
Porrentruy	76.1	35.3	9	7	90.57	26.6

	UrbanizationRate	FertilityCategory
Courtelary	83.0	High
Delemont	54.9	High
Franches-Mnt	60.3	High
Moutier	63.5	High
Neuveville	56.5	Medium
Porrentruy	64.7	Medium

Exercise i.2

- Filter observations where Fertility > 70
- Arrange results by Education in descending order

Solution

```

swiss_data |>
  filter(Fertility > 70) |>
  arrange(desc(Education))

```

	Fertility	Agriculture	Examination	Education	Catholic	Infant.Mortality
Neuveville	76.9	43.5	17	15	5.16	20.6
Sarine	82.9	45.2	16	13	91.38	24.4
Sion	79.3	63.1	13	13	96.83	18.1
Le Locle	72.7	16.7	22	13	11.22	18.9
Courtelary	80.2	17.0	15	12	9.96	22.2
Boudry	70.4	38.4	26	12	5.62	20.3
Delemont	83.1	45.1	6	9	84.84	22.2
Glane	92.4	67.8	14	8	97.16	24.9
Grandson	71.7	34.0	17	8	3.30	20.0

Payerne	74.2	58.1	14	8	5.23	23.8
Moutier	85.8	36.5	12	7	33.77	20.3
Porrentruy	76.1	35.3	9	7	90.57	26.6
Broye	83.8	70.2	16	7	92.85	23.6
Gruyere	82.4	53.3	12	7	97.67	21.0
Val de Ruz	77.6	37.6	15	7	4.97	20.0
Veveyse	87.1	64.5	14	6	98.61	24.5
Martigwy	70.5	78.2	12	6	98.96	19.4
Franches-Mnt	92.5	39.7	5	5	93.40	20.2
Paysd'enhaut	72.0	63.5	6	3	2.56	18.0
Monthey	79.4	64.9	7	3	98.22	20.2
Sierre	92.2	84.6	3	3	99.46	16.3
Conthey	75.5	85.9	3	2	99.71	15.1
Herens	77.3	89.7	5	2	100.00	18.3
Oron	72.5	71.2	12	1	2.40	21.0

	UrbanizationRate	FertilityCategory
Neuveville	56.5	Medium
Sarine	54.8	High
Sion	36.9	Medium
Le Locle	83.3	Medium
Courtelay	83.0	High
Boudry	61.6	Medium
Delemont	54.9	High
Glane	32.2	High
Grandson	66.0	Medium
Payerne	41.9	Medium
Moutier	63.5	High
Porrentruy	64.7	Medium
Broye	29.8	High
Gruyere	46.7	High
Val de Ruz	62.4	Medium
Veveyse	35.5	High
Martigwy	21.8	Medium
Franches-Mnt	60.3	High
Paysd'enhaut	36.5	Medium
Monthey	35.1	Medium
Sierre	15.4	High
Conthey	14.1	Medium
Herens	10.3	Medium
Oron	28.8	Medium

Exercise i.3

- Group data by FertilityCategory
- Calculate average Education for each category using group_by() and summarize()

Solution

```
swiss_data |>
  group_by(FertilityCategory) |>
  summarize(
    AverageEducation = mean(Education, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 3 x 2
```


	FertilityCategory	AverageEducation
	<fct>	<dbl>
1	Low	24.5
2	Medium	8.38
3	High	7.7

Exercise i.4

- Find the top 3 observations with highest Fertility values
- Display only Fertility and Education columns

Solution

```
swiss_data |>
  slice_max(Fertility, n = 3, with_ties = FALSE) |>
  select(Fertility, Education)
```

	Fertility	Education
Franches-Mnt	92.5	5
Glane	92.4	8
Sierre	92.2	3

Exercise i.5

- Create EducationToFertilityRatio = Education / Fertility
- Show all observations where the ratio is between 0.1 and 0.2

Solution

```
swiss_data |>
  mutate(EducationToFertilityRatio = Education / Fertility) |>
  filter(between(EducationToFertilityRatio, 0.1, 0.2))
```

	Fertility	Agriculture	Examination	Education	Catholic	Infant.Mortality
Courtelay	80.2	17.0	15	12	9.96	22.2
Delemont	83.1	45.1	6	9	84.84	22.2
Neuveville	76.9	43.5	17	15	5.16	20.6
Sarine	82.9	45.2	16	13	91.38	24.4
Aigle	64.1	62.0	21	12	8.52	16.5
Aubonne	66.9	67.5	14	7	2.27	19.1
Avenches	68.9	60.7	19	12	4.43	22.7
Grandson	71.7	34.0	17	8	3.30	20.0
Lavaux	65.1	73.0	19	9	2.84	20.0
Morges	65.5	59.8	22	10	5.23	18.0
Orbe	57.4	54.1	20	6	4.20	15.3
Payerne	74.2	58.1	14	8	5.23	23.8
Rolle	60.5	60.8	16	10	7.72	16.3
Yverdon	65.4	49.5	15	8	6.10	22.5
St Maurice	65.0	75.9	9	9	99.06	17.8
Sion	79.3	63.1	13	13	96.83	18.1
Boudry	70.4	38.4	26	12	5.62	20.3
La Chauxdfnd	65.7	7.7	29	11	13.79	20.5
Le Locle	72.7	16.7	22	13	11.22	18.9
ValdeTravers	67.6	18.7	25	7	8.65	19.5

	UrbanizationRate	FertilityCategory	EducationToFertilityRatio
Courtelary	83.0	High	0.1496259
Delemont	54.9	High	0.1083032
Neuveville	56.5	Medium	0.1950585
Sarine	54.8	High	0.1568154
Aigle	38.0	Medium	0.1872075
Aubonne	32.5	Medium	0.1046338
Avenches	39.3	Medium	0.1741655
Grandson	66.0	Medium	0.1115760
Lavaux	27.0	Medium	0.1382488
Morges	40.2	Medium	0.1526718
Orbe	45.9	Low	0.1045296
Payerne	41.9	Medium	0.1078167
Rolle	39.2	Medium	0.1652893
Yverdon	50.5	Medium	0.1223242
St Maurice	24.1	Medium	0.1384615
Sion	36.9	Medium	0.1639344
Boudry	61.6	Medium	0.1704545
La Chauxdfnd	92.3	Medium	0.1674277
Le Locle	83.3	Medium	0.1788171
ValdeTravers	81.3	Medium	0.1035503

Exercise i.6

- Group by FertilityCategory
- Calculate median Fertility and standard deviation of Education
- Store the results in a new data frame

Solution

```
swiss_stats <- swiss_data |>
  group_by(FertilityCategory) |>
  summarize(
    MedianFertility = median(Fertility, na.rm = TRUE),
    StandardDeviationEducation = sd(Education, na.rm = TRUE),
    .groups = "drop"
  )

swiss_stats
```

```
# A tibble: 3 x 3
  FertilityCategory MedianFertility StandardDeviationEducation
  <fct>              <dbl>              <dbl>
1 Low                55                14.2
2 Medium             69.3                5.95
3 High              84.8                3.02
```

Exercise i.7

- Use `n_distinct()` to determine the number of distinct values in Education and FertilityCategory

Solution

```
swiss_data |>
  summarize(
    DistinctEducation = n_distinct(Education),
    DistinctFertilityCategory = n_distinct(FertilityCategory)
  )
```

```
  DistinctEducation DistinctFertilityCategory
1                19                      3
```

Exercise j: visualizing data

Exercise j.1

- Load the dataset `swiss.csv` or call `data(swiss)`

Solution

```
data(swiss)
swiss_data <- swiss
```

Exercise j.2

- Print the last 3 observations of the dataset
- Create summary statistics for all variables

Solution

```
tail(swiss_data, 3)
```

	Fertility	Agriculture	Examination	Education	Catholic	Infant.Mortality
V. De Geneve	35.0	1.2	37	53	42.34	18.0
Rive Droite	44.7	46.6	16	29	50.43	18.2
Rive Gauche	42.8	27.7	22	29	58.33	19.3

```
summary(swiss_data)
```

Fertility		Agriculture		Examination		Education		Catholic	
Min.	:35.00	Min.	: 1.20	Min.	: 3.00	Min.	: 1.00	Min.	: 2.150
1st Qu.:	:64.70	1st Qu.:	:35.90	1st Qu.:	:12.00	1st Qu.:	: 6.00	1st Qu.:	: 5.195
Median	:70.40	Median	:54.10	Median	:16.00	Median	: 8.00	Median	:15.140
Mean	:70.14	Mean	:50.66	Mean	:16.49	Mean	:10.98	Mean	:41.144
3rd Qu.:	:78.45	3rd Qu.:	:67.65	3rd Qu.:	:22.00	3rd Qu.:	:12.00	3rd Qu.:	:93.125
Max.	:92.50	Max.	:89.70	Max.	:37.00	Max.	:53.00	Max.	:100.000

Infant.Mortality	
Min.	:10.80
1st Qu.:	:18.15
Median	:20.00
Mean	:19.94
3rd Qu.:	:21.70
Max.	:26.60

Exercise j.3

- Create new variables:
 - `UrbanizationRate = 100 - Agriculture`
 - `FertilityCategory`: 0-60 “Low”, 60-80 “Medium”, 80-100 “High”
- Hint: use `cut()`

Solution

```
swiss_data$UrbanizationRate <- 100 - swiss_data$Agriculture

swiss_data$FertilityCategory <- cut(
  swiss_data$Fertility,
  breaks = c(0, 60, 80, 100),
  labels = c("Low", "Medium", "High"),
  include.lowest = TRUE
)

head(swiss_data)
```

	Fertility	Agriculture	Examination	Education	Catholic	Infant.Mortality
Courtelary	80.2	17.0	15	12	9.96	22.2
Delemont	83.1	45.1	6	9	84.84	22.2
Franches-Mnt	92.5	39.7	5	5	93.40	20.2
Moutier	85.8	36.5	12	7	33.77	20.3
Neuveville	76.9	43.5	17	15	5.16	20.6
Porrentruy	76.1	35.3	9	7	90.57	26.6

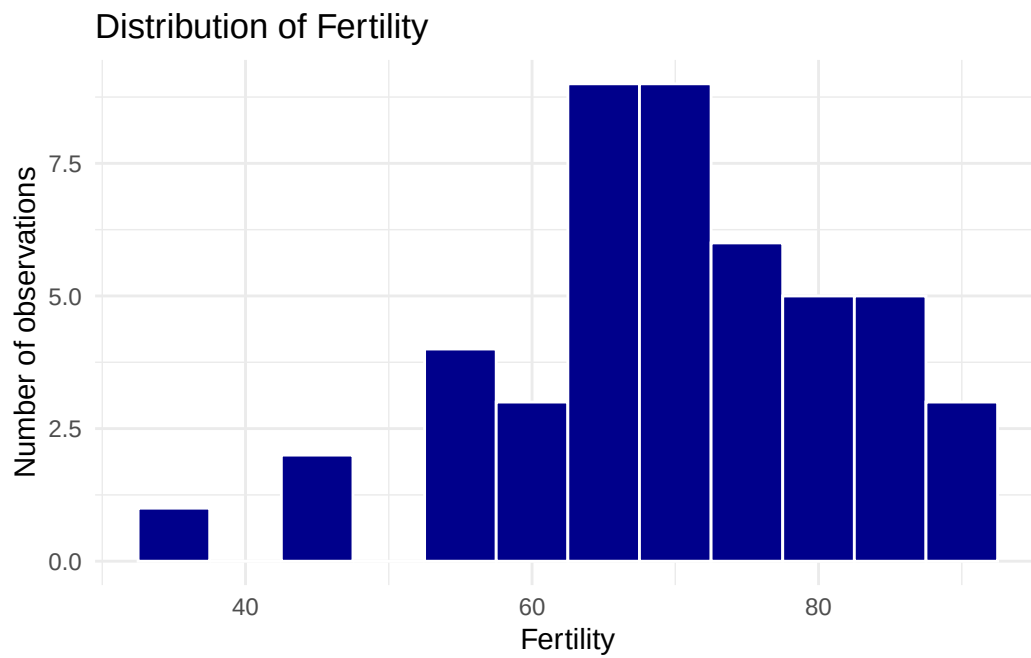
	UrbanizationRate	FertilityCategory
Courtelary	83.0	High
Delemont	54.9	High
Franches-Mnt	60.3	High
Moutier	63.5	High
Neuveville	56.5	Medium
Porrentruy	64.7	Medium

Exercise j.4

- Create a histogram of `Fertility`, fill bars in dark blue, add a minimal theme

Solution

```
ggplot(swiss_data, aes(x = Fertility)) +
  geom_histogram(binwidth = 5, fill = "darkblue", color = "white") +
  labs(title = "Distribution of Fertility",
       x = "Fertility", y = "Number of observations") +
  theme_minimal()
```

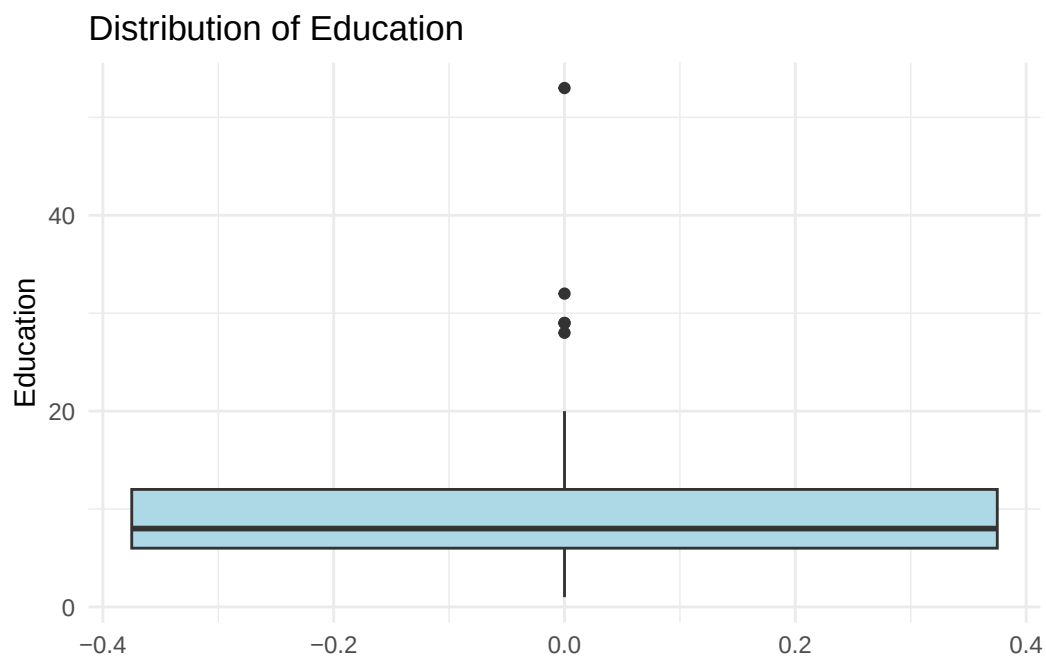


Exercise j.5

- Create a boxplot of the variable Education

Solution

```
ggplot(swiss_data, aes(y = Education)) +
  geom_boxplot(fill = "lightblue") +
  labs(title = "Distribution of Education", y = "Education") +
  theme_minimal()
```

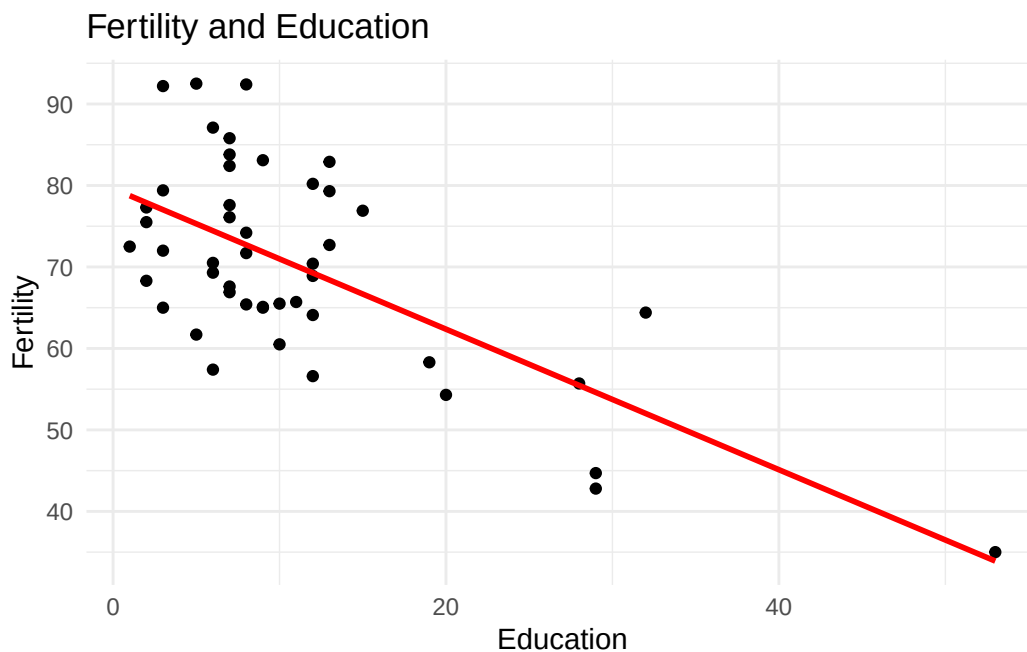


Exercise j.6

- Create a scatter plot of Education vs. Fertility
- Add a regression line (linear model)

Solution

```
ggplot(swiss_data, aes(x = Education, y = Fertility)) +  
  geom_point() +  
  geom_smooth(method = "lm", se = FALSE, color = "red") +  
  labs(title = "Fertility and Education",  
        x = "Education", y = "Fertility") +  
  theme_minimal()
```



Exercise j.7

- Create scatter plots of Agriculture vs. Examination
- Show them separately for each FertilityCategory using facet_wrap()

Solution

```
ggplot(swiss_data, aes(x = Agriculture, y = Examination)) +  
  geom_point() +  
  facet_wrap(~ FertilityCategory) +  
  labs(title = "Agriculture and Examination by Fertility Category",  
        x = "Agriculture", y = "Examination") +  
  theme_minimal()
```

Agriculture and Examination by Fertility Category

